DISCIPLINA: TESTE DE SOFTWARE

Refatoração de Testes e Detecção de Test Smells

Análise Estática com ESLint e Aplicação do Padrão AAA

Aluno: João Victor Filardi Souza Pinto

Matrícula: 806565

Repositório GitHub: https://github.com/JoaoFilardi/test-smelly

2026

1. Análise de Test Smells

A análise da suíte original (test/userService.smelly.test.js) revelou diversos Test Smells que tornavam os testes frágeis, obscuros e propensos a falsos positivos. Abaixo estão descritos os 3 principais Test Smells encontrados:

1.1. Lógica Condicional em Testes (Conditional Test Logic)

• Onde ocorre: No segundo teste ('deve desativar usuários se eles não forem administradores'), que utiliza um laço for combinado com uma estrutura condicional if/else, e no quarto teste ('deve falhar ao criar usuário menor de idade'), que envolve a chamada em um bloco try/catch.
• Por que é um mau cheiro: Testes unitários devem ser lineares, determinísticos e autoexplicativos. Estruturas de repetição e ramificações dentro de um teste aumentam a complexidade e ocultam qual fluxo realmente foi validado.
• Risco representado: Alto risco de falsos positivos (testes que passam mesmo com bug no sistema). No caso do try/catch, se o método createUser deixar de lançar exceção para menores de 18 anos, o fluxo nunca entra no bloco catch e o teste passa silenciosamente sem executar nenhuma asserção.

1.2. Teste Ansioso (Eager Test)

• Onde ocorre: No primeiro teste ('deve criar e buscar um usuário corretamente') e no segundo teste (que valida simultaneamente a desativação de usuário comum e o bloqueio de desativação para administrador).
• Por que é um mau cheiro: Um único caso de teste exercita múltiplos comportamentos e métodos distintos da classe UserService (como createUser e getUserById), misturando múltiplas ações (Act) e asserções (Assert).
• Risco representado: Viola o princípio de responsabilidade única nos testes e dificulta o diagnóstico rápido quando ocorre uma falha, pois não fica claro imediatamente qual regra ou método quebrou.

1.3. Teste Frágil (Fragile Test) e Teste Ignorado (Skipped Test)

• Onde ocorre: No terceiro teste ('deve gerar um relatório de usuários formatado'), que exige uma formatação textual rígida ('ID: ..., Nome: Alice, Status: ativo\n'), e no quinto teste deixado vazio com test.skip.
• Por que é um mau cheiro: O teste do relatório acopla a verificação à implementação exata da string em vez do comportamento essencial (presença dos dados dos usuários), enquanto o teste com test.skip deixa código incompleto e sem asserções.
• Risco representado: Qualquer ajuste cosmético no cabeçalho ou espaçamento do relatório quebra o teste sem que haja erro funcional (gerando manutenção desnecessária), e o teste ignorado reduz a cobertura real de cenários de borda.

2. Processo de Refatoração (Antes e Depois)

Abaixo é apresentado o comparativo de um dos testes mais problemáticos da suíte original (que acumulava Lógica Condicional e Eager Test) e como ele foi refatorado no novo arquivo test/userService.clean.test.js:

Antes (Código com Smells - test/userService.smelly.test.js):

```
test('deve desativar usuários se eles não forem administradores', () => {
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30);
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);
  const todosOsUsuarios = [usuarioComum, usuarioAdmin];

  for (const user of todosOsUsuarios) {
    const resultado = userService.deactivateUser(user.id);
    if (!user.isAdmin) {
      expect(resultado).toBe(true);
      const usuarioAtualizado = userService.getUserById(user.id);
      expect(usuarioAtualizado.status).toBe('inativo');
    } else {
      expect(resultado).toBe(false);
    }
  }
});
```

Depois (Código Limpo - test/userService.clean.test.js):

```
test('deve desativar um usuário comum com sucesso', () => {
  // Arrange
  const usuarioComum = userService.createUser('Comum', 'comum@teste.com', 30, false);

  // Act
  const resultado = userService.deactivateUser(usuarioComum.id);
  const usuarioAtualizado = userService.getUserById(usuarioComum.id);

  // Assert
  expect(resultado).toBe(true);
  expect(usuarioAtualizado.status).toBe('inativo');
});

test('não deve permitir desativar um usuário administrador', () => {
  // Arrange
  const usuarioAdmin = userService.createUser('Admin', 'admin@teste.com', 40, true);

  // Act
  const resultado = userService.deactivateUser(usuarioAdmin.id);
  const usuarioAtualizado = userService.getUserById(usuarioAdmin.id);

  // Assert
  expect(resultado).toBe(false);
  expect(usuarioAtualizado.status).toBe('ativo');
});
```

Explicação das decisões de refatoração: O teste original foi desmembrado em dois testes independentes e com nomes descritivos, eliminando completamente o laço for e a condicional if/else. Em ambos os testes foi aplicado rigorosamente o padrão Arrange, Act, Assert (AAA). Além disso, nos demais testes do arquivo limpo: (1) separou-se a criação e a busca de usuário em testes distintos; (2) substituiu-se o bloco try/catch pelo matcher expect(() => ...).toThrow(...); (3) desacoplou-se o teste de relatório da formatação exata, verificando apenas a presença dos dados essenciais; e (4) implementou-se o teste de relatório vazio que estava desativado com test.skip.

3. Relatório da Ferramenta (ESLint)

Na primeira execução do comando npx eslint . após configurar o arquivo .eslintrc.json com o plugin eslint-plugin-jest, a ferramenta identificou automaticamente 6 problemas (4 erros e 2 avisos) no arquivo original:



Como a ferramenta automatizou a detecção: O ESLint com eslint-plugin-jest inspeciona estaticamente a árvore sintática do código sem precisar executar os testes. A regra jest/no-conditional-expect detectou imediatamente os quatro usos de expect dentro de blocos condicionais (if/else nas linhas 44, 46 e 49, e catch na linha 73). Já as regras jest/no-disabled-tests e jest/expect-expect alertaram sobre o teste desativado (test.skip) e sem asserções na linha 77. Na validação final, o novo arquivo test/userService.clean.test.js passou pelo ESLint com 0 erros e 0 avisos, e todos os testes passaram com sucesso no comando npm test.

4. Conclusão

A prática demonstrou que uma suíte de testes que apenas passa na execução pode esconder falhas graves de projeto (Test Smells) capazes de mascarar bugs em produção. A aplicação disciplinada do padrão Arrange, Act, Assert (AAA) torna os testes curtos, legíveis, focados em comportamento e fáceis de manter. Em conjunto, ferramentas de análise estática como o ESLint atuam como uma barreira automatizada contínua, impedindo que testes frágeis, incompletos ou com lógica condicional sejam introduzidos no repositório, elevando a confiabilidade e a sustentabilidade do software a longo prazo.